Why Kubernetes Cannot Contain a Full Lazy-Pulling Cache: Two Impossibility Results and a Bounded, Honest Design for Model Serving

Georgii Kliukovkin

arXiv:2611.11739v1 [cs.DC] 9 Nov 2026

***Abstract*—Lazy container-image pulling makes cold starts of autoscaled model-serving fleets on Kubernetes nearly independent of model size, but it moves the model's bytes into a node-level cache that the platform neither bounds nor accounts for. This article establishes, by experiment, two impossibility results for that cache. A monitoring-side remedy cannot catch its failure: a sentinel readiness probe stays green while 272 of 280 files are unreadable. A platform-side remedy cannot contain it: kubelet, given full disk-pressure visibility, frees 0 bytes and evicts 26 bystander pods, because its reclamation actuator is keyed to an accounting that does not know the cache exists. The article then eliminates the failure class inside the component that owns the bytes, with no change to kubelet or the container runtime interface, by two design rules: a byte budget whose pressure signal is keyed to filesystem truth, so that a never-blocking, lossy accounting still enforces a hard bound (held against about 301,000 dropped updates), and pass-through semantics under which a failed cache write never becomes a failed read, with the coupling traced to a single `io.MultiWriter`. Across six pre-specified campaigns on AWS, upstream stargz-snapshotter fails 273 of 280 model files at 90% cache pre-fill; the proposed system reads all 150.3 GB with zero errors at +18.5% completion cost and holds the budget under a live read, co-tenancy, and a three-node fleet. A source audit shows the missing bound generalizes to SOCI and Nydus. The implementation is open source and has been proposed upstream.**

***Index Terms*—Caching, cloud computing, containers, Kubernetes, lazy loading, model serving, reliability, resource management.**

I. Introduction

SERVING a large model on Kubernetes, on a managed cloud service or a self-run cluster alike, begins with a delivery problem: hundreds of gigabytes of weights must reach the node before the first prediction. Eager image pulling pays that cost up front and scales linearly with size; lazy pulling—eStargz/stargz-snapshotter [3] and AWS SOCI [4]—mounts the image immediately and fetches content on demand, making cold time-to-first-prediction essentially independent of model size [2]. For autoscaled and scale-to-zero serving fleets, where every replica added under load pays the delivery cost before serving a single request, this is the difference between seconds and tens of minutes [1].

The price of that deferral is paid by a component most operators never configure and the platform does not account for: the snapshotter's node-level cache, shared by every lazily mounted image on the node, into which every fetched chunk is written. In prior work we characterized what happens when its volume fills under purely legitimate load [2]: pods that are already `Ready` and serving begin failing reads of their model files—67–94% of files at high residual occupancy—while the pod's readiness, health endpoint, and prediction endpoint remain green (for at least 196 s in our instrumented timeline). The failure is invisible to every management-plane signal Kubernetes exposes—readiness, liveness, prediction endpoints, node conditions, events—and observable only in snapshotter-specific telemetry that serving stacks do not collect by default. It is a fault with no symptom at the layer that owns remediation.

That measurement study answered *what happens*; this paper answers *what should*. It presents the design, implementation, and evaluation of bounded, honest lazy pulling: a rework of the lazy cache's contract with the platform above it, built as roughly 12,000 changed lines (about 40% tests) on stargz-snapshotter and proposed upstream. No result below depends on the under-review companion study: the vanilla arm of our evaluation re-establishes the failure on this paper's own rigs (Section V).

Why not just monitor better, or let the platform react? These are the two remediation routes a cluster operator actually has, monitoring and platform-driven eviction, and we close both empirically before building. First, detection-side mitigation is structurally limited: we built the natural mitigation—a readiness probe that reads sentinel files through the mount—and it reported green while 272 of 280 files were unreadable, because damage concentrates in file regions the probe's sample does not cover. A sampling probe converts silent corruption into a dice roll, not a guarantee. Second, platform-side remediation acts on the wrong accounting: with the cache partition exported to kubelet's image filesystem accounting and production eviction thresholds enabled, kubelet correctly raised `NodeHasDiskPressure`—and then freed 0 bytes, because containerd reports 741 KB of reclaimable image storage while 91.1 GiB of cache sits on disk, and fell through to evicting 26 pods that had never touched the partition. The platform is *sighted but powerless*: the gap is not signal; it is an actuator keyed to accounting that does not know the cache exists. Elimination must therefore happen inside the component that owns the bytes.

This paper makes four contributions:

- **Two measured impossibility results.** Monitoring-side and platform-side remediation, the two routes available to

G. Kliukovkin is an independent researcher in Sunnyvale, CA 94085 USA (e-mail: kliukovkin@gmail.com; ORCID 0009-0002-3323-1932).

Preprint. This manuscript is under review at IEEE Access (manuscript Access-2026-57465, submitted 4 November 2026).

an operator, are each shown by experiment to be unable to contain this failure class: a sentinel probe stays green while 272/280 files are unreadable, and kubelet with full pressure visibility frees nothing and evicts bystanders. These generalize to any node-local cache outside the platform's reclamation accounting (Sections II, VI).

- **A bounded cache that survives its own accounting.** Byte-budget eviction driven by a durable chunk index, with the pressure signal keyed to filesystem truth precisely because the index is lossy by contract: the filesystem decides *how much*, the index decides *which*. The budget held against ∼301,000 dropped accounting updates where a naïve rule died at ∼127,000 (Sections III, V).
- **An honest data path, with its root cause.** Pass-through semantics—a chunk that cannot be cached is served to its caller and simply not kept—sever the failure structurally; the coupling they sever is traced to a single `io.MultiWriter` on the fetch path, with a regression test that fails on the parent commit and passes on our fix (Sections III, IV).
- **Evaluation on AWS against the failure suite of our companion study, through tenancy and fleet scale.** The before/after is run on identical rigs: upstream fails 273/280 files at 90% pre-fill (three independent reproductions); our system serves 280/280 across the full 150.3 GB read at +18.5% completion cost, with the budget never exceeded while 224.6 GB is evicted under the read. We report an honest negative on eviction-policy choice, and the operational findings the road here surfaced—including that the shipped systemd unit silently defeats the upstream FUSE manager (Section V)—together with a source-level audit locating SOCI and both Nydus modes on the boundedness and coupling axes (Section II-C).

II. Background and the Failure Ledger

A. Lazy pulling and the node-level cache

An OCI image is a stack of compressed layer archives; an eager pull downloads and unpacks all of them before the container starts. Lazy-pulling snapshotters reformat (eStargz) or index (SOCI) the image so a FUSE filesystem can be mounted immediately and individual chunks fetched from the registry on first read. Every fetched chunk is written into a node-level on-disk cache shared by all lazily mounted images on the node: for stargz-snapshotter, an HTTP blob-chunk cache and a filesystem cache under one root. Model serving is the workload where both halves of this design are pushed hardest: artifacts reach hundreds of gigabytes, autoscaling multiplies cold starts, and a serving pod's correctness depends on every byte of its weights being readable long after `Ready`.

B. Five measured facts

The design in this paper is a response to five facts, each established by controlled measurement; none is hypothetical. F1–F2 are from our prior study [2]; F3–F5 from the pre-design measurement campaign for this work (rig details in Section V). Table I summarizes the ledger and the requirement each fact imposes on the design.

TABLE I
THE FAILURE LEDGER: EACH MEASURED FACT AND THE DESIGN REQUIREMENT IT IMPOSES. F1–F2 FROM [2]; F3–F5 FROM THIS WORK'S PRE-DESIGN CAMPAIGN.

	Measured fact	Key evidence	Requirement
F1	cache unbounded on disk	74 config keys, none bound bytes	bound it at all
F2	exhaustion corrupts *already-serving* pods	67–94% files fail; signals green ≥196 s	decouple cache failure from read failure
F3	sampling probes protect their sentinels, not the model	probe green at 272/280 unreadable	decouple by construction, not monitoring
F4	platform sighted but powerless	kubelet freed 0 B; evicted 26 innocent pods	bound must live where the bytes live
F5	shipped remediation defeats itself	7/7 KillMode trials; up to +18 GB per graceful restart	bound must survive restarts

F1: The cache is unbounded on disk. No configuration key bounds the on-disk size of stargz-snapshotter's cache tree: the two cache-sizing knobs bound in-memory LRU entries and open file descriptors. We verified the key surface of the development head is byte-identical to the release we measured—74 keys, none size-related. Cache directories are cleaned only per-layer when an image is removed.

F2: Exhaustion corrupts the already-serving. When the cache volume fills under sustained legitimate reads, running pods fail reads of model files—67–94% of files, scaling monotonically with residual occupancy—while readiness, health, and prediction endpoints stay green (196 s lower bound in an instrumented timeline). Damage concentrates in file *tails*: after one induction, every one of 280 files passed a 1 MiB read at offset zero while 262 failed a full read. Any health check that samples is therefore structurally biased toward the cached head of the data.

F3: Detection-side mitigation fails by sampling. We implemented the natural fix—a sentinel readiness probe that tail-reads a fixed set of files through the mount—and pre-specified its falsifier. It reported green while 272 of 280 files were unreadable: the probe protects its sentinels, not the model. This is F2's bias, weaponized against the monitor itself.

F4: The platform is sighted but powerless. We exported the cache partition to kubelet's image-filesystem accounting (the mechanism upstream added for disk-space visibility) and enabled production eviction thresholds, then filled the cache. The full chain fired: `NodeHasDiskPressure` was raised; image garbage collection ran—and freed **0 bytes**, reporting 741 KB of reclaimable images while 91.1 GiB of cache sat on disk, because the snapshotter's content cache is not "images" in containerd's accounting; kubelet then fell through to its only remaining actuator and evicted 26 pods that had never written to the partition, in a loop that cannot reclaim anything, while a fresh lazy pod stayed `Pending` indefinitely. The gap is not signal; it is an actuator keyed to the wrong accounting. Remediation must live in the component that owns the bytes.

F5: The shipped remediation defeats itself. stargz-snapshotter ships an opt-in FUSE manager whose purpose is to keep mounts alive across daemon restarts. In 7/7 pre-specified

trials, enabling it under the project's own systemd unit changed nothing: the unit's default control-group kill mode SIGTERMs the detached manager on every `systemctl restart`, and its SIGTERM handler unmounts everything (the manager escapes the process group, not the cgroup). The consequence is measured, not inferred: in the control arm every restart left running pods with dead mounts—8/8 model files failing with `ESTALE` from the first restart on. Under a corrected unit the manager works—and then each *graceful* restart duplicates cache contents in place, up to 18 GB per steady-state restart with zero application reads between restarts (node-disk usage from 10.0 GB to 77.8 GB over four restarts), where upstream documentation acknowledges duplication only for unexpected termination. Both findings are reported upstream with fixes.

Together the ledger defines the requirement: the cache must be bounded by the component that owns it (F1, F4), failure of the cache must be decoupled from failure of reads (F2, F3), and the bound must survive restarts and the accounting's own imperfections (F5).

C. Is the class shared? A source audit of SOCI and Nydus

The ledger is measured on one snapshotter. To locate the rest of the ecosystem in it, we audited soci-snapshotter and Nydus (nydusd plus nydus-snapshotter, in both its RAFS-userspace and EROFS-over-fscache modes) at pinned releases—statically, source only, every claim carrying a file-and-line citation in the released audit document. The result (Table II) is not a uniform "they all share it," and is more useful for that.

F1 generalizes. Neither SOCI (72 leaf configuration keys; the LRU knobs bound in-memory entries and descriptors, and LRU eviction never unlinks a cache file) nor Nydus bounds the on-disk bytes of its cache; the one partial exception, fscache mode, delegates the bound to the kernel's `cachefiles` culling—whole-filesystem watermarks, configured outside either project, with the snapshotter unable even to measure usage. Sharper than the missing knob: both projects ship a *dead* cache-management key—SOCI's `http_cache_type` is read by nothing, and nydus-snapshotter's `cache_manager.gc_period` is stored and never read (no ticker exists)—so the configuration surface advertises management that is not implemented.

F2 is a design axis, not a universal. The shared property is *placement*: whether the cache write sits on the delivery path. SOCI couples more strongly than stargz—the span is fetched in full into memory and then *discarded* when its cache write fails, surfacing as `EIO` for bytes that had arrived. Nydus's RAFS mode sits on the other end: the reader is filled from the fetched buffer while the cache write is dispatched asynchronously, so a persist failure costs a cache entry, never a read—a production existence proof that the decoupling this paper argues for is deployable (one deprecated option, `cache.compressed`, re-couples it). Its fscache mode makes the failure invisible instead: a populate error is logged and the kernel is acknowledged unconditionally; what the reader then observes is decided kernel-side and is not determinable statically.

F5 splits along a one-sentence design lesson. The two systems that duplicate cache across restarts (stargz, SOCI—the latter unconditionally: a fresh temporary directory per resolution, no startup sweep, and `SIGTERM` skips cleanup entirely) are exactly the two that name cache directories non-deterministically; the two that re-adopt (both Nydus modes) derive the cache path from the content identity.

III. Design

Three principles organize the system, implemented as roughly 12,000 changed lines on stargz-snapshotter (about 40% tests); all behavior is opt-in and off by default.

A. Durable, non-blocking accounting

A persistent index (bbolt) records every cached chunk—path, size, last observed access, in-flight pin—maintained write-through from the cache's data path via a bounded queue that *never blocks and never fails a cache operation*: updates are dropped, counted, when the queue is full. Recency lives in the index, not filesystem `atime` (fleet nodes mount `noatime`). On startup the index is validated (including a manual check of bbolt's meta pages, since a truncated database faults the process rather than erroring) and rebuilt by a background scan when missing or inconsistent—which is also how a restarted daemon re-adopts existing cache instead of accounting beside it (F5's growth).

Two consequences of "never block" are worth naming because the evaluation later turns on them. First, the queue is bounded (65,536 entries) and sheds load by dropping, so the index is *lossy by contract*: correctness of enforcement cannot be allowed to depend on its completeness (Section V-D tests exactly this). Second, because updates are asynchronous, the index may momentarily trail the filesystem in either direction; every consumer of the index is written against "advisory" semantics, and the invariants below are chosen so that staleness only ever costs a missed eviction candidate or a redundant refetch, never a wrong byte. The accounting costs +211 ns (+1.9%) on the cache-hit read path and +1.28 µs (+4.5%) on the write path in microbenchmarks.

B. Byte-budget eviction, keyed to filesystem truth

Eviction runs inside the index's single writer goroutine—no new locks on the data path. When pressure exceeds a high watermark, victims are selected by a pluggable policy over the index snapshot (pinned entries excluded) and removed until a low watermark holds; unlinking during concurrent reads is safe because POSIX keeps data alive under open descriptors, and a subsequently needed chunk simply refetches through the existing miss path.

Six invariants, fixed at design time and each carried by a specific mechanism rather than by care, define eviction's safety envelope:

I1 Never evict a pinned chunk. A pin covers the in-flight window between staging-file creation and rename-commit; the index is the only victim source and pinned entries are excluded from policy iteration.

TABLE II
SOURCE AUDIT AT PINNED RELEASES: DOES THE REST OF THE LAZY-PULLING ECOSYSTEM SHARE THE FAILURE LEDGER? STATIC ANALYSIS; EVERY CELL IS BACKED BY FILE:LINE CITATIONS IN THE RELEASED AUDIT DOCUMENT.

System (pinned)	On-disk byte bound?	Cache-write failure can fail a read?	Restart re-adoption?
stargz v0.18.2	no: 74 keys, none size-related (F1)	yes: writer coupling on the delivery path (F2)	no: duplicates in place, up to 18 GB per steady-state restart (F5)
SOCI v0.15.0	no: 72 leaf keys; LRU knobs bound in-memory entries and FDs; LRU eviction never unlinks a file	yes, more strongly: the fully fetched span is discarded when its cache write fails; surfaces as `EIO`	no, unconditionally: fresh temp dir per resolution, no startup sweep, `SIGTERM` skips cleanup
Nydus RAFS v2.4.5	no: no byte key; snapshotter `gc_period` is dead code	no (default): reader served from the fetched buffer, persist is asynchronous; deprecated `cache.compressed` re-couples	yes: content-derived path, persisted chunk map
Nydus fscache v2.4.5	external only: kernel `cachefiles` culling, outside the snapshotter's control or visibility	invisible: populate errors logged, kernel acknowledged unconditionally; reader-visible effect is kernel-side (not determinable statically)	yes: readiness rebuilt from the backing file's extents

I2 Eviction cannot race commit. A chunk enters the index only after its rename-commit, so a path not yet committed cannot be selected. The stale direction—committed but not yet applied to the index—only makes eviction *miss* a candidate, which is safe.

I3 Unlink during read is safe. Reads go through an open descriptor, and POSIX keeps unlinked data alive until last close; eviction therefore takes no read-side locks at all.

I4 Refetch is the existing miss path. The next read of an evicted chunk sees `ENOENT` and falls into the ordinary cache-miss branch; eviction adds no new code to the hot path, and its cost is a refetch, which the evaluation prices.

I5 Crash safety is inherited from the advisory index. A crash between unlink and index-delete leaves a dangling entry; the next read corrects it as a miss and the rebuild scan reconciles fully. No ordering constraints, no `fsync` on the eviction path.

I6 Self-protection is policy, not mechanism. A 140 GB sweep through an 80 GB budget must evict its own tail; whether it may also flush other images' resident hot sets is the eviction policy's business. The mechanism guarantees only watermark convergence.

The subtle question is what "pressure" is. The index under-counts by design whenever updates are dropped, and an eviction engine acting on an under-count re-creates F2 with extra steps: it stops evicting while the partition fills. Our first implementation had exactly this defect; a measurement campaign found it (Section V). The corrected rule: pressure is the index total plus a conservative price for every dropped update, raised further to the filesystem's own measurement of the cache's usage whenever the index is known to be lossy—and when eviction has surrendered every chunk it knows about while the filesystem still reports over-budget, the index itself is declared wrong and a rate-limited rescan rebuilds it, with eviction re-run against the result. The filesystem is authoritative for *how much*; the index only ever decides *which*.

C. Honesty: cache failure is never read failure

The load-bearing principle. On the fetch path, bytes stream from the registry simultaneously into the cache writer and the caller's buffer; in upstream code one `io.MultiWriter` couples them, so a cache-side write error stops delivery to a caller who needed nothing from the cache (Section IV). Our pass-through semantics: a cache write that fails—after one emergency eviction cycle and one retry at the repeatable points—fails *only the caching*. The fetched bytes are served, the chunk is simply not kept, a counter records the skip, and the very next chunk is offered to the cache again. Exhaustion becomes a latency degradation with a metric, never a correctness event. The residual honesty surface is deliberately modest: budget occupancy, evictions, skipped writes, and per-`errno` fetch failures as Prometheus series, plus a node-events agent; given F3, we explicitly do *not* offer a probe as a safety mechanism.

IV. IMPLEMENTATION

A. The root cause, and severing it

Upstream's fetch path streams a range response into the cache and the caller simultaneously: for each chunk, a cache writer is opened and the response body is copied through `io.MultiWriter(cacheWriter, callerBuffer)`. `io.MultiWriter` stops at the first writer that returns an error—so an `ENOSPC` raised by the cache at any of its three failure points (opening the staging file, the copy itself, or the commit rename) stops the same bytes from reaching a caller that needed nothing from the cache. In the commit case the caller's buffer is already complete and is discarded anyway. The error propagates through the FUSE layer as `EIO` on a read of a file whose bytes had, in fact, arrived. This is the entire mechanism behind F2: a full cache does not merely stop caching; it vetoes reads.

Fig. 1 shows the coupling and its severing, condensed from the actual patch. The fix wraps the cache writer in a *tolerant writer* that remembers its first failure and reports full progress, so the caller's copy always completes; the staging file is then aborted—never committed partially—and a per-`errno` counter records the skip.

Chunks that arrive on the wire but have no live caller are drained to keep the multi-chunk response body in sync. A caller that joined another caller's in-flight fetch and finds the result uncached retries a bounded number of times and then fetches its range directly: a read is never failed because the cache could not keep something. A regression test asserting "`ENOSPC` in the cache, bytes still served" fails on the parent commit with the original error string and passes on the fix.

The property that generalizes is placement, not the particular API. SOCI couples the same way with no `io.MultiWriter`

```
// UPSTREAM: one writer chain couples cache and caller.
w := io.Writer(cw)              // cache writer
if _, ok := fetched[chunk]; ok {
    w = io.MultiWriter(w, allData[chunk])
}
if _, err := io.CopyN(w, r, chunk.size()); err != nil {
    cw.Abort()
    return err   // ENOSPC in the cache fails the READ
}

// OURS: the cache may fail; the caller's copy cannot.
type tolerantWriter struct { w io.Writer; err error }

func (t *tolerantWriter) Write(p []byte) (int, error) {
    if t.err != nil { return len(p), nil }
    if _, err := t.w.Write(p); err != nil { t.err = err }
    return len(p), nil       // always claims full progress
}

cached := &tolerantWriter{w: cw}
var w io.Writer = cached
if dest != nil { w = io.MultiWriter(cached, dest) }
if _, err := io.CopyN(w, r, chunk.size()); err != nil {
    cw.Abort()
    return err   // the FETCH failed: nothing to serve
}
if cached.err != nil {       // cache failed, caller is
     whole
    cw.Abort()
    return b.skippedCaching(chunk, fetched, cached.err)
}
```

Fig. 1. The root cause and its severing (condensed from the patch to the chunk-caching path). Upstream, `io.MultiWriter` stops at the first failing writer, so a cache-side `ENOSPC` halts delivery to the caller mid-copy. With the tolerant wrapper, only a failure of the fetch itself can fail the read.

at all: the fully fetched span is discarded when its cache write fails (Section II-C). Nydus's RAFS mode shows the opposite placement shipping in production—reader served from the fetched buffer, persist asynchronous. The design question for any lazy-pulling cache is therefore not "does it use `MultiWriter`" but "is the cache write on the delivery path."

B. Hardening the restart path

Two defects found by our own measurement campaigns shaped the final implementation. First, the index handover on daemon reconnect originally opened the new index before releasing the old one; the newcomer could lose the database lock, silently leaving the node with a configured budget and no eviction. The fix orders release before construction and adds a bounded retry, with two deliberately distinct alarm series: a retry that *succeeded* means handover is racing (a bug to report), a retry that gave up means the node is running unenforced (an outage). Second, the pressure calculation of Section III exists because its naïve predecessor—trusting the index alone—was demonstrated to let the budget fail exactly when protection matters most (Section V-D).

C. Observability surface

The honesty principle demotes monitoring from safety mechanism to diagnosis, and the shipped surface reflects that. Prometheus series cover budget occupancy and configured budget, eviction counts and bytes by policy, skipped cache writes, dropped accounting updates, rescan runs, and blob-fetch failures by `errno`; a small node-events agent mirrors threshold crossings (budget exceeded, enforcement unavailable, rescan escalation) into Kubernetes events so they surface where operators already look. The sentinel-file probe of F3 ships only as an explicitly labeled smoke check: after its falsification we do not offer any sampling probe as a correctness signal, and the two handover-alarm series of the previous subsection distinguish "racing but recovered" from "running unenforced"—the alert that matters most, because an unenforced node looks healthy until F2.

D. Upstream status

The work is public as a branch on a fork of stargz-snapshotter. The pass-through fix of Section IV-A and the first observability slice (per-`errno` blob-fetch failure counter) are open upstream pull requests; the byte bound is proposed upstream as a design issue; the two operational findings of F5 are filed upstream with data and one-line fixes; and the SOCI coupling finding of Section II-C is filed with the SOCI project with file-and-line evidence. Nothing in this paper's evaluation depends on merge status.

V. Evaluation

A. Method

All experiments run on a two-host rig in one AWS availability zone (us-east-1): a registry host and a node host, both i4i.2xlarge (8 vCPU, 64 GB RAM, local NVMe), kind/Kubernetes v1.36, containerd 2.3.1, with the snapshotter's cache on a dedicated 92 GB NVMe partition and a configured budget of 80 GB (watermarks 0.95/0.85). The dedicated registry host keeps fetch latency constant across arms, and the dedicated cache partition means exhaustion experiments cannot sideload the OS volume. The workload is the failure suite of our companion study [2]: a 140 GB-class eStargz model image of 280 files with a data-path-coupled predictor, so a read error is observed as a serving failure, not inferred from logs. The rig uses kind on EC2 rather than a managed Kubernetes service; the F4 and elimination results do not depend on that choice. The mismatch F4 measures is structural (the snapshotter's cache tree is not part of containerd's image store on any node type, and kubelet's split image filesystem (KEP-4191) accounts for containerd's image store, not for the snapshotter's cache beside it), the kubelet eviction chain exercised is the same code a production node runs, and the elimination mechanism lives entirely in the snapshotter, below the control plane.

Expectations for every campaign were pre-specified: written protocols with pass and fail criteria, fixed before the rig for that campaign existed and released with the artifact. (We say pre-specified rather than pre-registered: no third-party registry was used.) Each trial leaves an evidence bundle (journals, metric snapshots, `df`/`du`/`lsof` samples, probe transcripts) collected immediately at trial end, a rule instituted after a watchdog teardown cost one campaign its artifacts. Invalidated trials are preserved and flagged, not deleted; all raw data, evidence bundles, and the harness are released. One methodological rule was itself measured into existence: identical builds differed by up to 16% in absolute completion time across rig instances

TABLE III
THE SIX EVALUATION CAMPAIGNS. EACH RAN AGAINST PRE-SPECIFIED EXPECTATIONS; TWO FALSIFIED OUR OWN IMPLEMENTATION, AND THE FIXES WERE RE-CONFIRMED UNDER THE SAME PROTOCOL.

	Pre-specified question	Outcome
I	Can the platform (kubelet) or the shipped FUSE manager already contain the failure?	No: F4 and F5 established (Section II-B).
II	Does pass-through eliminate the failure class? Does a sentinel probe suffice?	Elimination confirmed (Table IV); probe falsified (F3); live-budget sweep held (Fig. 2); found an index lock-handover race.
III	Does the budget survive its own lossy accounting?	No—naïve pressure rule died at ∼127,000 drops (Section V-D); handover race root-caused.
IV	Do the fixes hold under the original protocols?	Corrected pressure held at ∼301,000 drops; handover clean (0 retries); restart growth flattened (Fig. 3); policy arms gated at 4.7%.
V	Does a contended regime separate LRU from 2Q?	Gate failed (6.1%); reported only as consistent with the null.
VI	Does the bound survive a second tenant and fleet-level concurrency? Is registry egress linear in node count?	Zero read errors under both; interference modest and replicated (Section V-E); egress linear to within 0.1%; the slowdown clause falsified (k=3 ran *faster*, cause uninstrumented and reported as such); found: a single tenant's hot set occupies 3.4× its bytes.

while an interleaved same-session A/B of two different builds differed by 0.2%—so every comparison below is within-session, and no cross-instance absolute is quoted as a comparison.

The results below come from six campaigns run in sequence (Table III). Two of them falsified our own implementation—the naïve pressure rule and a lock-handover race—and the fixes were then confirmed under re-runs of the same pre-specified protocol. We report the campaign structure because the defects it caught are themselves part of the design argument (Sections III, V-D).

B. Elimination of the failure class

Table IV is the headline. At 90% cache pre-fill, upstream v0.18.2 reads 4.1 GB before the sweep collapses and fails 273 of 280 model files with `EIO`—a result reproduced with identical counts on three independent rig instances across three campaigns. Identical counts across independent instances are strong evidence against a timing race on this configuration and consistent with a structural mechanism—which files survive appears fixed by the geometry of chunk boundaries against the pre-fill (we compared counts, not file identities, so set-level determinism is inferred, not verified): identically provisioned nodes under this workload fail identically. Our system, on the same rig in the same session, reads the full 150.3 GB with **zero** errors, in both replicates. The 4,079–4,142 skipped cache writes per run confirm the out-of-space path was genuinely exercised—the zero-error result is pass-through working, not pressure failing to materialize. The price of honesty was replicated as five interleaved within-session pairs of the same full read at 0% versus 90% pre-fill (medians 953.4 s versus 1129.8 s): median **+18.5%**, range +15.3% to +19.0%, all ten runs 280/280 with zero errors. An earlier single pair on a different instance had measured +20.6%—outside the replicated range—so the ratio itself varies by instance, and we quote only the replicated median. Exhaustion has become a latency tax priced by registry proximity: on this rig's same-AZ registry it is +18.5%; against a distant registry every refetch pays the longer round trip, a cost we do not measure here.

TABLE IV
ELIMINATION RESULT: FULL READ OF A 140 GB-CLASS MODEL (280 FILES) AT 90% CACHE PRE-FILL, WITHIN-SESSION PAIRS, N=2 EACH. THE VANILLA ROW IS THE WITHIN-SESSION PAIR MEMBER FOR THE QUOTED RUNS; THE TWO FURTHER VANILLA REPRODUCTIONS ON OTHER INSTANCES (IDENTICAL COUNTS) ARE CORROBORATION, NOT COMPARISON ANCHORS.

	files OK	I/O errors	bytes read	writes skipped
vanilla v0.18.2	7 / 280	273	4.1 GB	0
ours	**280 / 280**	**0**	150.3 GB	4,079–4,142

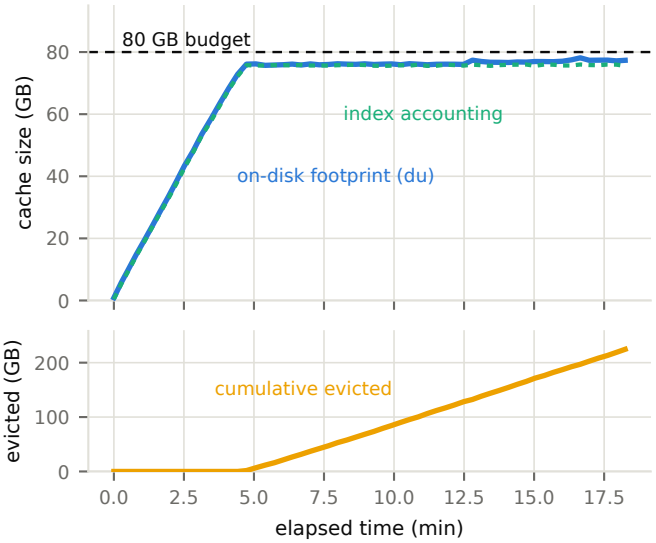

Fig. 2. An 80 GB budget under a live 140 GB sweep (59 samples over one full-model read). Top: on-disk cache footprint (`du`) and the index's own accounting converge just under the budget at the high watermark; index-reported peak 95.0% of budget, on-disk peak 97.7%. Bottom: 224.6 GB is evicted beneath the read—with zero read errors.

C. Bounded at full scale

Fig. 2 shows the budget under the full 140 GB sweep with an 80 GB budget: the engine evicted 224.6 GB beneath the live read—every byte of it refetchable, none of it vetoing a read—while index-reported occupancy peaked at 95.0% of budget (pinned at the high watermark) and the on-disk footprint peaked at 97.7%, never crossing the budget (59 samples; accounting drift versus `du` between −7.75% and +0.08% excluding the first, near-empty sample, where the asynchronous index trails a cold cache's opening seconds by design). Two transient blob-fetch errors early in the run were absorbed by the retry path and never surfaced: all 280 files read cleanly. The two occupancy curves in the figure are the design's split made visible: the index's accounting tracks the filesystem's own measurement closely under healthy operation, and the pressure rule of Section III exists for exactly the moments they diverge.

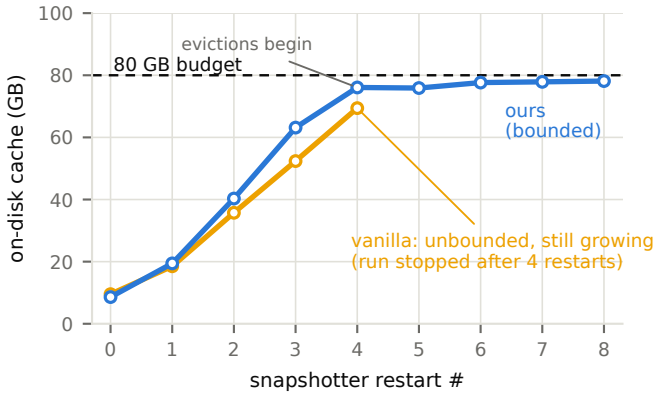

Fig. 3. Restart-driven cache duplication (F5), vanilla versus ours. The y-axis is the cache tree's on-disk footprint (`du`); the vanilla node's partition itself stood at 84% used when its run was stopped after four restarts. Vanilla with a working FUSE manager duplicates cache contents on every *graceful* restart with zero application reads in between; our re-adoption scan plus byte budget flattens the same pathology just under the budget, with zero read errors across eight restarts.

The restart pathology of F5 is capped by the same mechanism (Fig. 3): where the working FUSE manager grew the cache by up to 18 GB per graceful restart, unbounded (node-disk usage from 10.0 GB to 77.8 GB over four restarts—84% of its cache partition and still climbing when the run was stopped), our build's re-adoption scan plus budget flattens occupancy just under the budget from the fourth restart on, with zero read errors across eight restarts. A hypothesis that the duplication was deleted-but-open inodes was tested and refuted (`lsof` accounting showed 0 bytes at every step); the growth is consistent with duplicate chunk data on disk, which is precisely what a byte budget can reclaim regardless of its exact composition.

D. Surviving lossy accounting

The design's hardest test was self-inflicted. The accounting queue drops updates rather than block the data path; our first eviction engine trusted the resulting under-count, and a write-flood campaign killed it: at ∼127,000 dropped updates the partition reached 100% and the snapshotter could no longer start. The corrected pressure rule (Section III) was then driven with ∼301,000 dropped updates through the same flood: `du` held at 76 GB against the 80 GB budget, the node remained startable, and the escalation rescan fired 3–4 times per arm during enforcement (no arm ran with the rescan disabled, so its necessity is inferred from the design, not ablated). Table V contrasts the two rules under the same flood protocol; the arms ran in successive campaigns, but the comparison is categorical (fatal versus held) and the corrected rule absorbed 2.4× the drops that killed its predecessor, so cross-session placement cannot manufacture the result.

The known residual is bounded and reported: pressure under-estimated `du` by 12.3%–12.8% on 3 of ∼95 samples (chunks written during a rescan's own tree walk), never approaching the partition headroom above the budget on this geometry. The complement also held: with zero drops the filesystem is never consulted, so a shared partition's foreign bytes cannot be charged to the cache.

TABLE V
WRITE-FLOOD AGAINST THE BUDGET: THE SAME PROTOCOL, BEFORE AND AFTER THE PRESSURE RULE WAS KEYED TO FILESYSTEM TRUTH. "DROPS" ARE ACCOUNTING UPDATES DISCARDED BY THE NEVER-BLOCKING QUEUE.

Pressure rule	Drops absorbed	Outcome
index total only (naïve)	∼127,000	partition 100% full; snapshotter unstartable; fatal `ENOSPC`
+ drop estimate, raised to `statfs` when lossy; escalation rescan	∼301,000	`du` 76 GB vs. 80 GB budget; node startable; rescans fired 3–4×/arm; residual under-estimate 12.3%–12.8% on 3 of ∼95 samples

E. A second tenant, and a fleet

Two campaigns push past the single-pod, single-node envelope.

Interference under a shared budget. Pod A serves continuous reads from a 21.5 GB hot set while pod B sweeps the 140 GB-class image through the shared 80 GB budget (LRU, two repetitions). The headline survives tenancy: **zero** read errors on A across 4,100 measured reads, while B completes 280/280 within 0.4% of its single-tenant time. The interference is real, modest, and replicates: A's p50 rose +18.5% and +18.8% under the sweep (two repetitions agreeing to 0.3 pp) and its cache hit rate fell 5.2 and 13.3 points, attributed via per-blob registry accounting (A's image shares only a 2.3 MB base layer with B's, tallied separately). A pre-specified p99 rule proved under-powered at N=2—the baseline's own between-repetition p99 spread exceeds most of the effect—and is reported as undecided rather than leaned on; p50 and hit rate both replicate.

The finding we did not go looking for. In the baseline phase—pod A alone on the node—its 21.5 GB hot set occupied ∼73 GB of cache (3.4× its bytes), pinning occupancy at the high watermark and evicting continuously with no second tenant present (repetitions agree: 73.0 and 73.2 GB, ∼610,000 evictions per 300 s window). A single read of the set costs 2.0×—the known duplication across the HTTP-chunk and filesystem cache tiers—and the remaining inflation accrues with re-reads; this campaign did not isolate which tier dominates, or whether the growth is stale copies or accounting. Effective capacity for this workload shape is roughly budget/3.4. This reorders I6's question: before asking whether a sweep may flush *another* tenant's hot set, a tenant whose hot set is 27% of the nominal budget already cannot keep it. Both interference numbers above are therefore measured on top of an already-thrashing tenant, not against a quiescent warm baseline; the pre-specified "zero refetches in steady state" entry condition was unsatisfiable on this geometry and is reported as such rather than silently relaxed.

Registry amplification at $k = 3$. Three nodes, each with its own 80 GB budget at 90% pre-fill, read the full image simultaneously (start skew ≤20 ms) from one registry host. Every node's budget held with zero errors (2,520 reads). Aggregate registry egress was linear in k to within +0.0%/−0.1%

of 3× the same-session k=1 baseline: lazy pulling under exhaustion multiplies registry load with no cross-node relief, as it must with node-local caches and no peer-to-peer layer. The registry's measured 10.3 Gbps NIC reached only 29% at k=3, and the pre-specified expectation that concurrent readers would slow each node was falsified: all six comparisons ran 1.1%–5.6% *faster* than the same node's own k=1 run. We hypothesize registry-side page-cache warmth across rounds and did not instrument it; the claims this campaign licenses are the linearity, the held budgets, and the unsaturated NIC. The saturation point lies beyond three nodes on this hardware and is not extrapolated.

F. Eviction policy: an honest negative

The mechanism takes pluggable policies, and serving workloads are scan-shaped, so we expected scan-resistant eviction (a 2Q-style probationary/protected split: a chunk must be touched twice before it earns protection from a passing sweep) to beat LRU. It did not. The comparison is disciplined by a gate, pre-specified for the definitive runs though instituted mid-investigation: before any between-policy difference may be interpreted, every arm's warm baseline run—same build, policy irrelevant—must agree within 5%, since a difference between identical configurations is the rig's noise floor speaking. The gate exists because a first attempt produced an apparent 2Q win that was traced to a baseline mismatch and withdrawn. Under the gate, eight interleaved arm-runs (four per policy) shared a warm baseline to within 4.7%, and under the pre-specified effect-size rule—a between-policy difference counts only if it exceeds the spread between repetitions of the same policy—LRU and 2Q were not separable at p50, p95, p99, or hit rate. With the noise floor at 4.7%, this campaign's minimum detectable effect is roughly 5%: we claim equivalence within that bound, not the absence of any effect. A follow-up in a deliberately contended regime (70 GB hot set against the 80 GB budget, ∼10.5 M evictions per arm) failed its own gate (6.1%) and is reported only as consistent with the null. Across the entire campaign no measurement produced positive evidence for 2Q. We ship LRU as the default and report the policy interface as mechanism, not advantage.

G. Overheads

Accounting costs +211 ns (+1.9%) per cache-hit read and +1.28 µs (+4.5%) per cache write in microbenchmarks; the filesystem-truth pressure path added 0.2% in an interleaved same-session A/B at full sweep scale. A 30 s race-detector stress run served 3.18 M reads while 231,638 chunks were evicted underneath with zero incorrect bytes and zero non-miss errors.

VI. Discussion: What Cloud Platforms Should Take

Three lessons generalize beyond stargz-snapshotter, and one objection deserves a direct answer.

Align the actuator with the accounting, or own the bytes. F4 is an instance of a general hazard: a platform whose reclamation actuator (kubelet image GC, pod eviction) is keyed to one accounting (containerd images, pod ephemeral storage) cannot manage a resource that lives outside it, no matter how good its signal is. Wherever a node component maintains a shared on-disk cache—snapshotters, CSI drivers, model caches—either its bytes must appear in an accounting the platform can act on, or the component must bound itself. We chose the second because it needs no kubelet or CRI changes; the first is the longer-term fix and worth a platform-level design discussion.

Cache failure must be decoupled from data failure by construction, not by monitoring. F3 shows the detection route is sampling-limited even when built in good faith; the pass-through result shows the construction route is cheap (+18.5% at total exhaustion, 0.2% when healthy). We suggest this as a design test for any read-through cache on a critical data path: inject a full cache volume and ask whether reads fail. The audit of Section II-C grades the ecosystem against it: Nydus RAFS passes by construction, stargz and SOCI fail, and fscache mode cannot be graded from user space.

Truth for *how much* and truth for *which* can come from different sources. An accounting that must never block the data path will be lossy; an enforcement decision that trusts it inherits the loss at the worst moment. Splitting the roles—filesystem-level usage decides when to evict, the index decides what—kept the budget standing under ∼301,000 dropped updates. The same split should apply to any advisory index guarding a hard resource.

Sidestepping the cache does not dissolve the lesson. An operator can avoid this failure class by not routing weights through the image at all—volume-based model storage, or the OCI image volume source. Those designs trade away what lazy pulling is for (registry-native distribution, provenance, and the cold-start result that motivates this line of work [1]) and move the same question elsewhere: whatever component then caches model bytes on the node faces the same bound-and-honesty test. Likewise the folk remedy—a periodic prune of the cache tree—is an unsynchronized actuator with no accounting: it cannot respect in-flight pins or recency, deletes at arbitrary points in the write path, and bounds nothing between prunes. We did not evaluate it, but nothing in it addresses the write-failure coupling that produces F2.

VII. Related Work

Lazy image distribution. Slacker [5] established lazy container distribution; DADI [6], Starlight [7], FaaSNet [8], Nydus [9], eStargz [3], and SOCI [4] developed formats and fetching architectures. That literature evaluates start-up acceleration; our companion study [2] measured its deferred cost and failure semantics, and this paper closes the loop with a fix. **Bounded caches for remote filesystems.** CernVM-FS has long shipped an LRU-evicted, quota-bounded local cache for a FUSE-mounted remote filesystem [10]; we adopt the spirit and differ in what forced the design: a Kubernetes node's cache is multi-tenant across pods, its accounting must never block a serving data path (hence lossy accounting and filesystem-truth pressure, Section V-D), and its failure mode couples into

pod lifecycle semantics rather than a single user's job. **Cache replacement.** The policy interface implements LRU and a 2Q-style split [13]; scan-resistant designs from ARC [14] to the recent S3-FIFO [15] and SIEVE [16] target exactly the access patterns where LRU is weakest. Our negative result does not contradict that literature: on a serving node dominated by large sequential sweeps through a budget smaller than the working set, the gains scan resistance buys on skewed web workloads have nothing to grip—no policy can rescue a sweep that must evict its own tail (I6). **Mount survival across daemon restarts** is prior art we adopt, not claim: Nydus's failover [9] and JuiceFS's smooth upgrade [20] both hand off live FUSE sessions (the FUSE data path's costs are themselves well characterized [17]); stargz-snapshotter's own FUSE manager does the same—F5's finding is that its shipped packaging silently defeats it, and its graceful-restart cache duplication was undocumented. **Kubernetes resource management and failure semantics.** Node-pressure eviction and image GC [11] manage disk the platform can see; F4 measures precisely what happens when they meet a resource they cannot reclaim. The green-while-failing pods of F2 and kubelet's unreclaiming eviction loop of F4 are instances of patterns documented at scale—fail-slow rather than fail-stop behavior [18] and sustained metastable degradation [19]; our contribution is tracing one such loop to an actuator/accounting mismatch and removing the trigger by construction. Work on LLM serving cold starts (e.g., ServerlessLLM [12]) attacks load-time from the checkpoint side and is complementary: our concern is what the delivery cache does to a fleet after `Ready`. **Management of lazy pulling and Kubernetes fleets.** Chen *et al.* [21] manage lazy pulling across edge nodes by sharing image caches collaboratively to shorten start-up; KubeTwin [22] models Kubernetes deployments as digital twins to predict their behavior at scale. Both treat the node-level cache as a resource to be placed and sized; neither asks what the platform observes, or can do, once that cache exhausts its volume, which is the management gap this paper measures and closes.

VIII. Limitations

The failure mechanism is node-local, and the multi-node campaign measured three same-AZ nodes: registry saturation (k^*), cross-AZ registries, and placement consequences of budget pressure (a scheduler declining to place lazy pods on saturated caches) remain unevaluated, and the $k{=}3$ speed-up's suspected registry page-cache cause is uninstrumented. The two-tenant campaign measured interference on top of an already-thrashing tenant (Section V-E); its p99 rule was under-powered at N=2, and the 3.4× cache inflation behind the thrash—which tier dominates, stale copies versus accounting—is the paper's most important open measurement. One hardware generation (i4i NVMe); absolute times scale with the rig, which is why only within-session comparisons are quoted, and the +18.5% price of exhaustion is specific to a same-AZ registry. The elimination result is established on the same failure suite that documented the problem; the claim that the class is closed for other workload shapes rests on the mechanism argument of Section IV-A—the severed coupling is the only path by which a cache write error reached the reader—together with the `ENOSPC` regression test, not on workload breadth. The write-flood result bounds the budget's survival at ∼301,000 dropped updates but does not establish the rate at which reconciliation stops converging; that ceiling is unmeasured. The policy negative is established on one gate-passing workload family (four arm-runs per policy) with a contended follow-up that failed its own gate; it licenses "no observed advantage exceeding the ∼5% minimum detectable effect," not "no possible advantage." The 12.3%–12.8% reconciliation residual was measured on one partition geometry with 12 GB of partition headroom above the budget; tighter geometries deserve their own check. The cross-snapshotter audit is static: SOCI's and Nydus's behavior under runtime exhaustion—including whether SOCI shares the sampling-blindness of F2's damage pattern—was not measured here, and four specific questions the sources cannot answer are flagged as such in the released audit document; the managed lazy-pulling offerings of cloud providers (GKE Image Streaming, Azure Artifact Streaming, SOCI on EKS and Fargate) are closed-source beyond the SOCI component and could not be audited. The budget is a single per-node bound: per-tenant or per-namespace quotas, fairness, and noisy-neighbor isolation are not provided, and the 3.4× inflation finding makes them the natural next step for the policy interface. The $k{=}3$ simultaneous exhausted reads are the closest this evaluation comes to a scale-out burst of concurrent cold starts under budget pressure; an explicit autoscaling campaign remains to be run. Cost modelling—translating the measured +18.5% completion cost and linear registry egress into provider prices—is out of scope; the quantities needed for it are reported.

IX. Conclusion

Lazy pulling's cold-start win for model serving was never in doubt; its safety was. We traced the silent corruption of already-serving pods to a single coupling on the fetch path, severed it with pass-through semantics, and bounded the cache with an eviction engine that survives its own lossy accounting by trusting the filesystem for how much and the index for which. On the failure suite that documented the problem, the failure class is gone: 280/280 files served across a 150.3 GB read at 90% pre-fill, against 7/280 upstream, for a +18.5% completion cost and data-path overheads of +211 ns per read and +1.28 µs per write. The platform above cannot do this on the cache's behalf—given perfect visibility it freed nothing and evicted bystanders—so the bound belongs where the bytes live. Harness, raw data, evidence bundles, and the implementation are open source at https://github.com/kliukovkin/lazy-model-delivery-study and the stargz-snapshotter fork referenced there.

Acknowledgment

The author discloses that AI-based assistants (Anthropic Claude) were used in developing the implementation, benchmark tooling, and analysis scripts and in editing the manuscript text; no section of this article was generated by an AI system.

The study design, the experiments, validation of all results, and responsibility for the content rest with the author. All raw data, per-trial evidence bundles, pre-specified protocols, and the measurement harness are publicly available at https://github.com/kliukovkin/lazy-model-delivery-study (paper3/). The author declares no competing interests and received no funding for this work.

REFERENCES

[1] G. Kliukovkin, "Cold-start model delivery in Kubernetes inference serving: an empirical study of OCI-based distribution and its integrity," *IEEE Access*, vol. 14, 2026, doi: 10.1109/ACCESS.2026.3731156.

[2] G. Kliukovkin, "The lazy pod that lies: deferred cost and failure semantics of lazy container image pulling for model serving on Kubernetes," 2026, arXiv:2608.19412.

[3] containerd project, "stargz-snapshotter: fast container image distribution plugin with lazy pulling," https://github.com/containerd/stargz-snapshotter, v0.18.2, 2026.

[4] AWS Labs, "soci-snapshotter: a containerd snapshotter plugin for lazy loading with the Seekable OCI format," https://github.com/awslabs/soci-snapshotter, 2026.

[5] T. Harter, B. Salmon, R. Liu, A. C. Arpaci-Dusseau, and R. H. Arpaci-Dusseau, "Slacker: fast distribution with lazy Docker containers," in *Proc. 14th USENIX FAST*, 2016, pp. 181–195.

[6] H. Li *et al.*, "DADI: block-level image service for agile and elastic application deployment," in *Proc. USENIX ATC*, 2020.

[7] J. L. Chen, D. Liaqat, M. Gabel, and E. de Lara, "Starlight: fast container provisioning on the edge and over the WAN," in *Proc. USENIX NSDI*, 2022, pp. 35–50.

[8] A. Wang *et al.*, "FaaSNet: scalable and fast provisioning of custom serverless container runtimes at Alibaba Cloud Function Compute," in *Proc. USENIX ATC*, 2021.

[9] Dragonfly project, "Nydus: the Dragonfly image service," https://github.com/dragonflyoss/nydus, 2026.

[10] J. Blomer, P. Buncic, and T. Fuhrmann, "CernVM-FS: delivering scientific software to globally distributed computing resources," in *Proc. 1st Int. Workshop Netw.-Aware Data Manage. (NDM)*, 2011, pp. 49–56.

[11] Kubernetes Documentation, "Node-pressure eviction," https://kubernetes.io/docs/concepts/scheduling-eviction/node-pressure-eviction/, 2026.

[12] Y. Fu *et al.*, "ServerlessLLM: low-latency serverless inference for large language models," in *Proc. USENIX OSDI*, 2024, pp. 135–153.

[13] T. Johnson and D. Shasha, "2Q: a low overhead high performance buffer management replacement algorithm," in *Proc. 20th Int. Conf. Very Large Data Bases (VLDB)*, 1994, pp. 439–450.

[14] N. Megiddo and D. S. Modha, "ARC: a self-tuning, low overhead replacement cache," in *Proc. 2nd USENIX FAST*, 2003, pp. 115–130.

[15] J. Yang, Y. Zhang, Z. Qiu, Y. Yue, and R. Vinayak, "FIFO queues are all you need for cache eviction," in *Proc. 29th ACM SOSP*, 2023, pp. 130–149.

[16] Y. Zhang, J. Yang, Y. Yue, Y. Vigfusson, and K. V. Rashmi, "SIEVE is simpler than LRU: an efficient turn-key eviction algorithm for web caches," in *Proc. 21st USENIX NSDI*, 2024, pp. 1229–1246.

[17] B. K. R. Vangoor, V. Tarasov, and E. Zadok, "To FUSE or not to FUSE: performance of user-space file systems," in *Proc. 15th USENIX FAST*, 2017, pp. 59–72.

[18] H. S. Gunawi *et al.*, "Fail-slow at scale: evidence of hardware performance faults in large production systems," in *Proc. 16th USENIX FAST*, 2018, pp. 1–14.

[19] N. Bronson, A. Aghayev, A. Charapko, and T. Zhu, "Metastable failures in distributed systems," in *Proc. 18th Workshop Hot Topics Oper. Syst. (HotOS)*, 2021, pp. 221–227.

[20] Juicedata, "JuiceFS: a distributed POSIX file system built on top of object storage," https://github.com/juicedata/juicefs, 2026.

[21] C.-C. Chen, Y. Chiang, Y.-C. Lee, and H.-Y. Wei, "Edge computing management with collaborative lazy pulling for accelerated container startup," *IEEE Trans. Netw. Service Manag.*, vol. 21, no. 6, pp. 6437–6450, Dec. 2024.

[22] D. Borsatti *et al.*, "KubeTwin: a digital twin framework for Kubernetes deployments at scale," *IEEE Trans. Netw. Service Manag.*, vol. 21, no. 4, pp. 3889–3903, Aug. 2024.

Georgii Kliukovkin is an independent researcher in Sunnyvale, CA, USA. His work concerns container image distribution, model-serving infrastructure on Kubernetes, and the dependability of cloud-native systems, and includes merged contributions to the KServe project (OCI-based model storage) and contributions proposed to containerd/stargz-snapshotter. He holds a master's degree.